

INSTITUTO FEDERAL SUL-RIO-GRANDENSE
CAMPUS PASSO FUNDO
CURSO DE CIÊNCIA DA COMPUTAÇÃO

RELATÓRIO DE ANÁLISE CRÍTICA DA
ARQUITETURA DE MICROSERVIÇOS:
CENÁRIO STREAMFLOW

PASSO FUNDO
2026

Equipe técnica:

Gabrieli Granja Brandalise

Luigi Mathew Scortegagna

Luanny Rodrigues Dutra

RESUMO

Este relatório técnico apresenta, de forma estruturada, a análise crítica da arquitetura de microsserviços da plataforma StreamFlow, uma aplicação de streaming de vídeo composta por múltiplos serviços independentes em produção. A avaliação foi realizada a partir da execução local do sistema com Docker Compose, análise da documentação arquitetural e investigação de evidências obtidas por meio de logs e arquivos gerados durante os testes.

São descritos os principais problemas identificados na arquitetura, incluindo custo operacional elevado, alta latência em fluxos síncronos, fragilidade na observabilidade, inconsistência de dados entre serviços, desalinhamento organizacional conforme a Lei de Conway e uso inadequado de microsserviços em determinados componentes. Para cada um desses pontos, são apresentadas evidências práticas, diagnóstico das causas raiz e propostas de melhoria com base em princípios de arquitetura de software.

O documento também propõe ajustes estruturais, como a consolidação de serviços com baixo grau de autonomia, adoção de comunicação assíncrona para reduzir acoplamento, implementação de estratégias de observabilidade distribuída e reorganização da equipe conforme domínios de responsabilidade. Além disso, foi realizada uma atividade complementar utilizando o framework Molecular para substituir parte das chamadas síncronas entre serviços por comunicação orientada a eventos. Dessa forma, busca-se tornar a arquitetura mais eficiente, sustentável e alinhada à capacidade operacional da equipe.

Palavras-chave: Microsserviços, Arquitetura de Software, Observabilidade, Consistência Eventual, Docker Compose, Lei de Conway.

LISTA DE TABELAS

Tabela 1 – Dados Overhead Operacional.....	7
Tabela 2 – Níveis de Consistência por Domínio no StreamFlow	13

LISTA DE ILUSTRAÇÕES

Figura 1 – Log do analytics-service na Geração de Relatório	8
Figura 2 – Log do Teste do Play sem <i>notification-service</i>	8
Figura 3 – Retorno da API	9
Figura 4 – Serviço recommendation-service entra no fluxo de Play	11
Figura 5 – Serviço catalog-service Verifica Licença no Fluxo	11
Figura 6 – Log Gateway	13
Figura 7 – Log que Confirma o Estado Degradado no Gateway	14
Figura 8 – Log dos conteúdos do Catálogo e o Campo license_active	16
Figura 9 – Método GET para Verificar Licença no Catálogo	16
Figura 10 – Método GET de Histórico de Recomendações.....	16
Figura 11 – Emissão do evento playback.started no streaming-service	21
Figura 12 – Processamento do evento playback.started no recommendation-service.....	21

SUMÁRIO

1	CONTEXTUALIZAÇÃO DA ATIVIDADE	6
2	CUSTO OPERACIONAL OCULTO	6
2.1	Visão Geral da Arquitetura	6
2.2	Estimativa de Overhead Operacional	7
2.3	Análise da Necessidade dos Serviços	7
2.4	Ações Recomendadas	10
3	LATÊNCIA DE REDE	10
3.1	Análise do Fluxo de Execução	10
3.2	Evidências de Latência	10
3.3	Causa Raíz e Proposta de Melhoria	12
4	OBSERVABILIDADE FRÁGIL	12
4.1	Análise do Estado Atual	12
4.2	Evidência do Problema	13
4.3	Causa Raíz e Lacunas de Observabilidade	1
4.4	Proposta de Melhoria	14
5	CONSISTÊNCIA EVENTUAL	15
5.1	Análise do Estado Atual	15
5.2	Exemplo e Causa da Inconsistência	15
5.3	Proposta de Melhoria e Níveis de Consistência	16
6	LEI DE CONWAY	17
6.1	Análise do Cenário Atual	17
6.2	Impacto na Distribuição do Conhecimento	18
6.3	Proposta de Reorganização e Ações Recomendadas	18
7	O CASO PRIME VIDEO	19
7.1	Análise do Cenário no StreamFlow	19
7.2	Causa Raíz	19
7.3	Proposta de Melhoria	19
8	REFATORAÇÃO DO FLUXO DE PLAY COM MOLECULER	20
8.1	Introdução ao Moleculer e Motivação	20
8.2	Implementação dos Eventos	20
8.3	Avaliação do Desempenho	22
8.4	Limitações da Abordagem	22

1. Contextualização da Atividade

A arquitetura de microsserviços consiste em um modelo no qual aplicações são estruturadas como um conjunto de serviços independentes, que se comunicam entre si por meio de APIs, permitindo maior modularidade e flexibilidade no desenvolvimento de sistemas (Red Hat, 2023).

Neste contexto, a atividade proposta consiste na análise crítica da arquitetura do StreamFlow, um sistema de streaming de vídeo composto por múltiplos microsserviços em produção. A avaliação foi realizada a partir da execução local do ambiente utilizando Docker Compose, ferramenta que permite definir e gerenciar múltiplos containers de forma integrada, facilitando a orquestração de serviços interdependentes em uma mesma aplicação (Hostinger, 2026), bem como da análise da documentação e dos fluxos arquiteturais disponibilizados.

Durante os testes, foram executados cenários de depuração com o objetivo de identificar falhas, gargalos e limitações da arquitetura. Como forma de evidência, foram coletados dados em arquivos JSON e logs, armazenados na pasta "evidencias/", os quais serviram de base para as análises apresentadas nas seções seguintes.

2. Custo Operacional Oculto

"Temos mais gente mantendo infraestrutura do que escrevendo features."

A afirmação do gerente é parcialmente correta. A arquitetura da StreamFlow é composta por 7 microsserviços em produção, além de um API Gateway, totalizando 8 unidades técnicas independentes. Embora alguns serviços possuam justificativa clara para isolamento, outros aumentam a complexidade sem entregar benefícios proporcionais.

2.1 Visão Geral da Arquitetura

Atualmente, o sistema é composto pelas seguintes unidades: *gateway*, *auth-service*, *catalog-service*, *streaming-service*, *recommendation-service*, *notification-service*, *billing-service* e *analytics-service*. Cada uma dessas unidades opera de forma independente, com seu próprio ciclo de execução e configuração.

2.2 Estimativa de Overhead Operacional

O overhead operacional da arquitetura pode ser resumido na Tabela 1:

Tabela 1 – Dados Overhead Operacional

Categoria	Quantidade	Descrição
Unidades Operacionais	8	Gateway + 7 microserviços
Dockerfiles	8	Um para cada serviço e um para o gateway
Arquivos <i>package.json</i>	8	Dependências isoladas por serviço
Containers/processos	8	Executados separadamente via Docker Compose
Pipelines de CI/CD	Até 8	Em um ambiente de produção, cada serviço poderia ter seu próprio pipeline de build, teste e deploy
Portas/configurações internas	8	Gateway na porta 8080 e serviços internos nas portas 3001 a 3007
Bancos de dados (SQLite)	5	<i>auth.db</i> , <i>catalog.db</i> , <i>streaming.db</i> , <i>recommendation.db</i> , <i>shared_billing.db</i>
Serviço sem banco	1	<i>notification-service</i>
Banco compartilhado	1 caso crítico	<i>billing-service</i> e <i>analytics-service</i> compartilham <i>shared_billing.db</i>

Fonte: Do autor

A partir desses dados, observa-se que a arquitetura impõe um custo operacional elevado, principalmente considerando o número de serviços em relação ao tamanho da equipe.

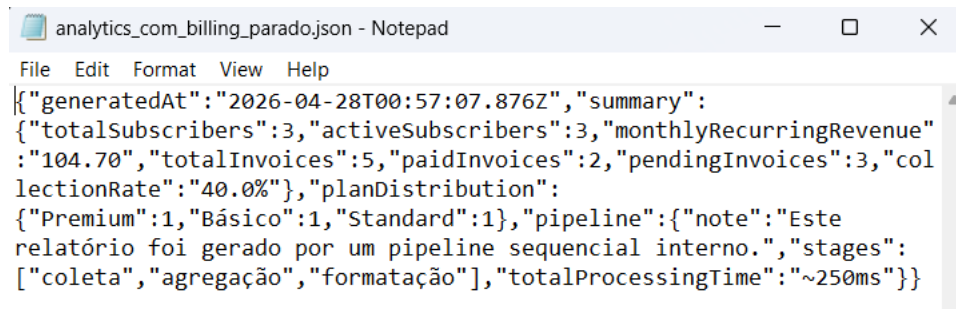
2.3 Análise da Necessidade dos Serviços

Nem todos os microserviços presentes na arquitetura possuem o mesmo nível de justificativa para existirem de forma independente. Alguns representam domínios bem definidos e críticos, enquanto outros aumentam a complexidade sem oferecer benefícios proporcionais. Entre os serviços com separação justificada, destacam-se o *auth-service*, responsável pela autenticação de usuários e emissão de tokens (JWT), sendo um componente sensível do ponto de vista de segurança; o *catalog-service*, que gerencia o catálogo de conteúdos e regras de licenciamento; o *streaming-service*, que controla as sessões de reprodução e é central para a funcionalidade principal da plataforma; e o *billing-service*, que concentra as regras de negócio relacionadas ao domínio financeiro.

Esses serviços possuem responsabilidades claras e isolamento que contribui para organização e segurança do sistema.

Por outro lado, existem serviços cuja independência não se justifica plenamente. O *analytics-service* não apresenta autonomia real. Evidências mostram que ele acessa diretamente o banco de dados compartilhado (*shared_billing.db*), sem depender da API do *billing-service*. No cenário de teste apresentado na Figura 1, o relatório financeiro foi gerado com sucesso mesmo com o *billing-service* indisponível. O resultado inclui métricas como número total de assinantes, receita recorrente mensal (*monthlyRecurringRevenue* = 104.70), distribuição de planos e status de faturas.

Figura 1 – Log do analytics-service na Geração de Relatório



```

analytics_com_billing_parado.json - Notepad
File Edit Format View Help
{"generatedAt":"2026-04-28T00:57:07.876Z","summary":
{"totalSubscribers":3,"activeSubscribers":3,"monthlyRecurringRevenue"
:"104.70","totalInvoices":5,"paidInvoices":2,"pendingInvoices":3,"col
lectionRate":"40.0%"},"planDistribution":
{"Premium":1,"Básico":1,"Standard":1},"pipeline":{"note":"Este
relatório foi gerado por um pipeline sequencial interno.,"stages":
["coleta","agregação","formatação"],"totalProcessingTime":"~250ms"}}

```

Fonte: Do autor

Além disso, o próprio retorno explicita que o processamento ocorre por meio de um pipeline interno, conforme o trecho:

```

"pipeline": {
  "note": "Este relatório foi gerado por um pipeline sequencial interno.",
  "stages": ["coleta", "agregação", "formatação"],
  "totalProcessingTime": "~250ms"
}

```

O *notification-service* também apresenta separação questionável. Ele não possui persistência de dados e atua apenas como um simulador de envio de notificações. No teste *play_sem_notification.json*, a reprodução ocorreu normalmente mesmo com falha nesse serviço. Evidências do log *logs_streaming_sem_notification.txt* (Figura 2) mostram que, durante a execução do endpoint */streaming/play*, o *streaming-service* registrou uma falha ao tentar se comunicar com o *notification-service*.

Figura 2 – Log do Teste do Play sem *notification-service*



```

logs_streaming_sem_notification.txt - Notepad
File Edit Format View Help
streaming-service-1 | {"level":30,"time":1777337710614,"pid":1,"hostname":"93b3519ab31e","msg":"Server listening at http://127.0.0.1:3003"}
streaming-service-1 | {"level":30,"time":1777337710615,"pid":1,"hostname":"93b3519ab31e","msg":"Server listening at http://172.18.0.2:3003"}
streaming-service-1 | {"level":30,"time":1777337710615,"pid":1,"hostname":"93b3519ab31e","msg":"streaming-service rodando na porta 3003"}
streaming-service-1 | {"level":30,"time":1777337724280,"pid":1,"hostname":"93b3519ab31e","reqId":"req-1","req":
{"method":"POST","url":"/streaming/play","host":"streaming-service:3003","remoteAddress":"172.18.0.9","remotePort":36428},"msg":"incoming request"}
streaming-service-1 | {"level":30,"time":1777337724282,"pid":1,"hostname":"93b3519ab31e","movieId":3,"userId":"user_1","msg":"Iniciando fluxo de
play..."}
streaming-service-1 | {"level":40,"time":1777337726891,"pid":1,"hostname":"93b3519ab31e","err":{"type":"TypeError","message":"fetch failed:
getaddrinfo ENOTFOUND notification-service","stack":"TypeError: fetch failed\n    at node:internal/deps/undici/undici:14976:13\n    at
process.processTicksAndRejections (node:internal/process/task_queues:103:5)\n    at async Object.<anonymous> (/app/index.js:90:5)\n    ncaused by:
Error: getaddrinfo ENOTFOUND notification-service\n    at GetAddrInfoReqWrap.onlookupall [as oncomplete] (node:dns:122:26)"},"msg":"Falha ao enviar
notificação"}
streaming-service-1 | {"level":30,"time":1777337726908,"pid":1,"hostname":"93b3519ab31e","reqId":"req-1","res":
{"statusCode":201,"responseTime":2627.614431999624,"msg":"request completed"}

```

Fonte: Do autor

Apesar disso, a requisição foi concluída com sucesso, retornando status 201 e tempo de resposta de aproximadamente 2627ms:

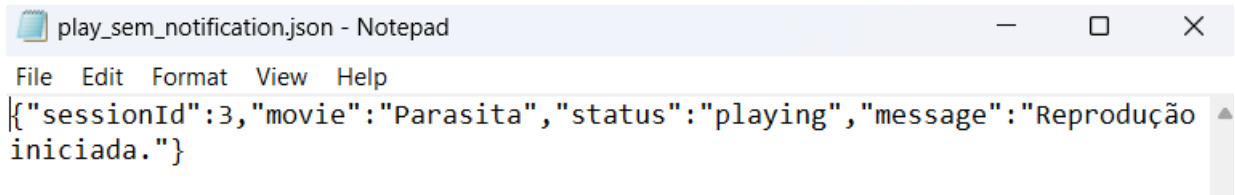
```

"statusCode":201,
"responseTime":2627ms,
"msg":"request completed"

```

E o retorno da API confirma que a reprodução foi iniciada normalmente (Figura 3):

Figura 3 – Retorno da API



```
play_sem_notification.json - Notepad
File Edit Format View Help
{"sessionId":3,"movie":"Parasita","status":"playing","message":"Reprodução iniciada."}
```

Fonte: Do autor

Esse conjunto de evidências demonstra que o envio de notificações não é uma pré-condição para o início da reprodução. Ou seja, o *notification-service* não impacta diretamente a funcionalidade principal do sistema, e sua presença no fluxo síncrono apenas aumenta a latência e a complexidade operacional, sem gerar benefício perceptível ao usuário.

Dessa forma, observa-se que parte da arquitetura foi decomposta além do necessário, resultando em serviços que não agregam autonomia significativa, mas aumentam o custo e a complexidade do sistema.

2.4 Ações Recomendadas

Diante da análise realizada, recomenda-se uma simplificação seletiva da arquitetura, mantendo a separação apenas nos serviços que apresentam ganho real de autonomia e organização. Nesse sentido, serviços como *auth-service*, *catalog-service*, *streaming-service* e *billing-service* devem ser preservados como unidades independentes, pois representam domínios críticos com responsabilidades bem definidas. Por outro lado, o *analytics-service* deve ser consolidado ao *billing-service*, uma vez que já depende diretamente do mesmo banco de dados e não apresenta autonomia suficiente para justificar sua existência como microsserviço. Da mesma forma, o *notification-service* deve ser removido do fluxo síncrono de execução e reestruturado para operar de forma assíncrona, sendo acionado por eventos como *PlaybackStarted*. Com essas alterações, a StreamFlow tende a reduzir significativamente o overhead operacional, melhorar a eficiência da equipe e manter uma arquitetura mais coerente com os princípios de microsserviços, aplicando a separação apenas onde ela agrega valor real.

3. Latência de Rede

“Um clique no Play toca 12 serviços antes do vídeo iniciar.”

A afirmação do gerente é parcialmente correta. Embora o fluxo atual não envolva literalmente 12 serviços, o processo de reprodução percorre múltiplas chamadas síncronas antes de responder ao usuário, o que aumenta a latência e introduz pontos adicionais de falha.

3.1 Análise do Fluxo de Execução

O fluxo de execução do endpoint *POST /streaming/play* envolve diversas interações entre serviços. De forma simplificada, o caminho percorrido é:

Cliente → API Gateway → *streaming-service* → *catalog-service* → *recommendation-service* → *notification-service* → criação da sessão → resposta ao cliente.

Considerando apenas as chamadas síncronas principais, o fluxo possui pelo menos 5 hops:

1. Cliente → Gateway
2. Gateway → *streaming-service*
3. *streaming-service* → *catalog-service*
4. *streaming-service* → *recommendation-service*
5. *streaming-service* → *notification-service*

Dentro do *streaming-service*, são realizadas 3 chamadas síncronas adicionais antes da resposta final, o que contribui diretamente para o aumento da latência.

3.2 Evidências de Latência

Os testes realizados evidenciam o impacto da cadeia síncrona no tempo de resposta. No arquivo *tempos_latencia.txt*, foram registrados os seguintes valores:

- GET /api/catalog: 0.007135s
- POST /api/streaming/play: 0.259294s

Isso representa uma diferença de aproximadamente 0.252s, sendo o endpoint de play cerca de 36 vezes mais lento que uma consulta simples ao catálogo.

Além disso, logs do sistema confirmam o impacto de serviços não essenciais no tempo de resposta. No arquivo *logs_streaming_sem_notification.txt*, o *streaming-service* registra falha ao tentar se comunicar com o *notification-service*, mas ainda assim conclui a requisição com sucesso (status 201), com tempo de resposta de aproximadamente 2627ms. Isso demonstra que o serviço de notificação não é essencial, mas ainda assim contribui para o aumento da latência.

Também há evidência da participação do *recommendation-service* no fluxo. No arquivo *logs_recommendation.txt* (Figura 4), observa-se uma requisição para */recommendations/viewed*, com tempo de aproximadamente 16,86ms, sendo executada de forma síncrona (uso de *await* no código), o que obriga o *streaming-service* a aguardar sua conclusão.

Figura 4 – Serviço recommendation-service entra no fluxo de Play



```
logs_recommendation.txt - Notepad
File Edit Format View Help
recommendation-service-1 | {"level":30,"time":1777337710765,"pid":1,"hostname":"9a5193fdffb3","msg":"Server listening at http://127.0.0.1:3004"}
recommendation-service-1 | {"level":30,"time":1777337710766,"pid":1,"hostname":"9a5193fdffb3","msg":"Server listening at http://172.18.0.8:3004"}
recommendation-service-1 | {"level":30,"time":1777337710766,"pid":1,"hostname":"9a5193fdffb3","msg":"recommendation-service rodando na porta 3004"}
recommendation-service-1 | {"level":30,"time":1777337724315,"pid":1,"hostname":"9a5193fdffb3","reqId":"req-1","req":{"method":"POST","url":"/recommendations/viewed","host":"recommendation-service:3004","remoteAddress":"172.18.0.2","remotePort":44056},"msg":"incoming request"}
recommendation-service-1 | {"level":30,"time":1777337724332,"pid":1,"hostname":"9a5193fdffb3","reqId":"req-1","res":{"statusCode":200,"responseTime":16.860757999587804,"msg":"request completed"}}
recommendation-service-1 | {"level":30,"time":1777337980194,"pid":1,"hostname":"9a5193fdffb3","reqId":"req-2","req":{"method":"GET","url":"/health","host":"recommendation-service:3004","remoteAddress":"172.18.0.9","remotePort":54926},"msg":"incoming request"}
recommendation-service-1 | {"level":30,"time":1777337980195,"pid":1,"hostname":"9a5193fdffb3","reqId":"req-2","res":{"statusCode":200,"responseTime":0.5545490002259612,"msg":"request completed"}}
recommendation-service-1 | {"level":30,"time":1777338071660,"pid":1,"hostname":"9a5193fdffb3","msg":"Server listening at http://127.0.0.1:3004"}
recommendation-service-1 | {"level":30,"time":1777338071661,"pid":1,"hostname":"9a5193fdffb3","msg":"Server listening at http://172.18.0.8:3004"}
recommendation-service-1 | {"level":30,"time":1777338071661,"pid":1,"hostname":"9a5193fdffb3","msg":"recommendation-service rodando na porta 3004"}
recommendation-service-1 | {"level":30,"time":1777338075984,"pid":1,"hostname":"9a5193fdffb3","reqId":"req-1","req":{"method":"GET","url":"/health","host":"recommendation-service:3004","remoteAddress":"172.18.0.9","remotePort":37998},"msg":"incoming request"}
recommendation-service-1 | {"level":30,"time":1777338075988,"pid":1,"hostname":"9a5193fdffb3","reqId":"req-1","res":{"statusCode":200,"responseTime":3.490815999917686,"msg":"request completed"}}
recommendation-service-1 | {"level":30,"time":1777338282599,"pid":1,"hostname":"9a5193fdffb3","msg":"Server listening at http://127.0.0.1:3004"}
recommendation-service-1 | {"level":30,"time":1777338282600,"pid":1,"hostname":"9a5193fdffb3","msg":"Server listening at http://172.18.0.8:3004"}
recommendation-service-1 | {"level":30,"time":1777338282600,"pid":1,"hostname":"9a5193fdffb3","msg":"recommendation-service rodando na porta 3004"}
recommendation-service-1 | {"level":30,"time":1777337710706,"pid":1,"hostname":"8b8a63124be9","msg":"Server listening at http://127.0.0.1:3002"}
catalog-service-1 | {"level":30,"time":1777337710707,"pid":1,"hostname":"8b8a63124be9","msg":"Server listening at http://172.18.0.4:3002"}
catalog-service-1 | {"level":30,"time":1777337710707,"pid":1,"hostname":"8b8a63124be9","msg":"catalog-service rodando na porta 3002"}
catalog-service-1 | {"level":30,"time":1777337724301,"pid":1,"hostname":"8b8a63124be9","reqId":"req-1","req":{"method":"GET","url":"/catalog/3/license","host":"catalog-service:3002","remoteAddress":"172.18.0.2","remotePort":45936},"msg":"incoming request"}
catalog-service-1 | {"level":30,"time":1777337724305,"pid":1,"hostname":"8b8a63124be9","reqId":"req-1","res":{"statusCode":200,"responseTime":4.3571290001273155,"msg":"request completed"}}
catalog-service-1 | {"level":30,"time":1777337980187,"pid":1,"hostname":"8b8a63124be9","reqId":"req-2","req":{"method":"GET","url":"/health","host":"catalog-service:3002","remoteAddress":"172.18.0.9","remotePort":43404},"msg":"incoming request"}
catalog-service-1 | {"level":30,"time":1777337980187,"pid":1,"hostname":"8b8a63124be9","reqId":"req-2","res":{"statusCode":200,"responseTime":0.48165900027379394,"msg":"request completed"}}
catalog-service-1 | {"level":30,"time":1777338008517,"pid":1,"hostname":"8b8a63124be9","reqId":"req-3","req":{"method":"GET","url":"/health","host":"catalog-service:3002","remoteAddress":"172.18.0.9","remotePort":43404},"msg":"incoming request"}
catalog-service-1 | {"level":30,"time":1777338008518,"pid":1,"hostname":"8b8a63124be9","reqId":"req-3","res":{"statusCode":200,"responseTime":0.8132299999706447,"msg":"request completed"}}
catalog-service-1 | {"level":30,"time":1777338075975,"pid":1,"hostname":"8b8a63124be9","reqId":"req-4","req":{"method":"GET","url":"/health","host":"catalog-service:3002","remoteAddress":"172.18.0.9","remotePort":43404},"msg":"incoming request"}
catalog-service-1 | {"level":30,"time":1777338075976,"pid":1,"hostname":"8b8a63124be9","reqId":"req-4","res":{"statusCode":200,"responseTime":0.6046190001070499,"msg":"request completed"}}
catalog-service-1 | {"level":30,"time":1777338162694,"pid":1,"hostname":"8b8a63124be9","reqId":"req-5","req":{"method":"GET","url":"/health","host":"catalog-service:3002","remoteAddress":"172.18.0.9","remotePort":58642},"msg":"incoming request"}
catalog-service-1 | {"level":30,"time":1777338162694,"pid":1,"hostname":"8b8a63124be9","reqId":"req-5","res":{"statusCode":200,"responseTime":0.5633910000324249,"msg":"request completed"}}
```

Fonte: Do autor

Por outro lado, a chamada ao *catalog-service*, evidenciada no arquivo *logs_catalog.txt* (Figura 5) de aproximadamente 4,36ms, é justificável, pois valida a existência e a licença do conteúdo antes da reprodução.

Figura 5 – Serviço catalog-service Verifica Licença no Fluxo.



```
logs_catalog.txt - Notepad
File Edit Format View Help
catalog-service-1 | {"level":30,"time":1777337710706,"pid":1,"hostname":"8b8a63124be9","msg":"Server listening at http://127.0.0.1:3002"}
catalog-service-1 | {"level":30,"time":1777337710707,"pid":1,"hostname":"8b8a63124be9","msg":"Server listening at http://172.18.0.4:3002"}
catalog-service-1 | {"level":30,"time":1777337710707,"pid":1,"hostname":"8b8a63124be9","msg":"catalog-service rodando na porta 3002"}
catalog-service-1 | {"level":30,"time":1777337724301,"pid":1,"hostname":"8b8a63124be9","reqId":"req-1","req":{"method":"GET","url":"/catalog/3/license","host":"catalog-service:3002","remoteAddress":"172.18.0.2","remotePort":45936},"msg":"incoming request"}
catalog-service-1 | {"level":30,"time":1777337724305,"pid":1,"hostname":"8b8a63124be9","reqId":"req-1","res":{"statusCode":200,"responseTime":4.3571290001273155,"msg":"request completed"}}
catalog-service-1 | {"level":30,"time":1777337980187,"pid":1,"hostname":"8b8a63124be9","reqId":"req-2","req":{"method":"GET","url":"/health","host":"catalog-service:3002","remoteAddress":"172.18.0.9","remotePort":43404},"msg":"incoming request"}
catalog-service-1 | {"level":30,"time":1777337980187,"pid":1,"hostname":"8b8a63124be9","reqId":"req-2","res":{"statusCode":200,"responseTime":0.48165900027379394,"msg":"request completed"}}
catalog-service-1 | {"level":30,"time":1777338008517,"pid":1,"hostname":"8b8a63124be9","reqId":"req-3","req":{"method":"GET","url":"/health","host":"catalog-service:3002","remoteAddress":"172.18.0.9","remotePort":43404},"msg":"incoming request"}
catalog-service-1 | {"level":30,"time":1777338008518,"pid":1,"hostname":"8b8a63124be9","reqId":"req-3","res":{"statusCode":200,"responseTime":0.8132299999706447,"msg":"request completed"}}
catalog-service-1 | {"level":30,"time":1777338075975,"pid":1,"hostname":"8b8a63124be9","reqId":"req-4","req":{"method":"GET","url":"/health","host":"catalog-service:3002","remoteAddress":"172.18.0.9","remotePort":43404},"msg":"incoming request"}
catalog-service-1 | {"level":30,"time":1777338075976,"pid":1,"hostname":"8b8a63124be9","reqId":"req-4","res":{"statusCode":200,"responseTime":0.6046190001070499,"msg":"request completed"}}
catalog-service-1 | {"level":30,"time":1777338162694,"pid":1,"hostname":"8b8a63124be9","reqId":"req-5","req":{"method":"GET","url":"/health","host":"catalog-service:3002","remoteAddress":"172.18.0.9","remotePort":58642},"msg":"incoming request"}
catalog-service-1 | {"level":30,"time":1777338162694,"pid":1,"hostname":"8b8a63124be9","reqId":"req-5","res":{"statusCode":200,"responseTime":0.5633910000324249,"msg":"request completed"}}
```

Fonte: Do autor

3.3 Causa Raíz e Proposta de Melhoria

A principal causa da latência está na mistura entre pré-condições obrigatórias e efeitos colaterais tratados como operações síncronas. O *streaming-service* aguarda a execução de todas as chamadas externas antes de responder ao usuário, mesmo quando algumas dessas operações não são essenciais para iniciar a reprodução. Enquanto a verificação de licença no *catalog-service* é necessária para garantir que o conteúdo pode ser reproduzido, ações como o registro de visualização no *recommendation-service* e o envio de notificações pelo *notification-service* são efeitos colaterais que não deveriam bloquear o fluxo principal.

Evidências dos testes reforçam esse diagnóstico: o sistema continua funcionando corretamente mesmo com falhas no *notification-service*, e o *recommendation-service* é acionado de forma síncrona sem ser uma dependência crítica. Isso demonstra que parte da latência observada é consequência de decisões arquiteturais que tratam operações secundárias como obrigatórias.

Diante disso, recomenda-se a redução da cadeia síncrona do fluxo de reprodução, mantendo apenas as operações essenciais no caminho crítico. O *streaming-service* deve validar a licença junto ao *catalog-service*, criar a sessão de reprodução e responder imediatamente ao usuário. As demais ações devem ser desacopladas por meio de comunicação assíncrona, com a publicação de eventos como *PlaybackStarted*, permitindo que serviços como *recommendation-service* e *notification-service* processem suas responsabilidades em segundo plano.

Nesse modelo, o fluxo passa a ocorrer da seguinte forma: o cliente realiza a requisição ao gateway, que a encaminha ao *streaming-service*; este, por sua vez, consulta o *catalog-service* para validação da licença, cria a sessão de reprodução e responde ao usuário. Após essa resposta, um evento é publicado, possibilitando que o *recommendation-service* registre a visualização e o *notification-service* envie notificações de forma assíncrona.

Com essa abordagem, o usuário deixa de aguardar a execução de serviços não críticos, fazendo com que o processo de *play* dependa apenas da verificação de licença e da criação da sessão local. Como resultado, o tempo de resposta deixa de ser impactado por operações secundárias, reduzindo significativamente a latência percebida e melhorando a eficiência geral do sistema.

4. Observabilidade Frágil

“Tivemos 3 incidentes e não sabemos nem onde foi.”

A afirmação do gerente é correta. Embora o StreamFlow possua mecanismos básicos de observabilidade, como logs estruturados e endpoints de *health check*, esses recursos são insuficientes para diagnosticar incidentes de forma confiável em um ambiente distribuído.

4.1 Análise do Estado Atual

Atualmente, os serviços utilizam logs estruturados contendo informações como método HTTP, URL, *statusCode*, *responseTime* e *reqId*, conforme evidenciado nos arquivos *logs_gateway.txt*, *logs_streaming.txt*, *logs_catalog.txt*, *logs_recommendation.txt* e *logs_analytics.txt*. Embora isso represente uma boa prática inicial, há uma limitação crítica: os logs não estão correlacionados entre os serviços.

O identificador *reqId* presente nos logs é gerado localmente por cada serviço, não sendo propagado ao longo da requisição. Dessa forma, uma mesma chamada pode aparecer com identificadores iguais em diferentes serviços, sem garantia de que pertencem à mesma requisição original. Isso impede o rastreamento ponta a ponta do fluxo.

4.2 Evidência do Problema

A limitação da observabilidade pode ser observada na análise dos logs. No arquivo *logs_gateway.txt* apresentado na Figura 6, o gateway registra uma requisição *POST /api/streaming/play*, encaminha a chamada para */streaming/play* e finaliza com status 201, com tempo total de aproximadamente 2649ms. No entanto, esse log não fornece informações sobre o que ocorreu internamente nos demais serviços.

Figura 6 – Log Gateway.



```

logs_gateway.txt - Notepad
File Edit Format View Help
gateway-1 | [00:55:11] [32mINFO[39m (1): [36mServer listening at http://127.0.0.1:8080[39m
gateway-1 | [00:55:11] [32mINFO[39m (1): [36mServer listening at http://172.18.0.9:8080[39m
gateway-1 | [00:55:11] [32mINFO[39m (1): [36mStreamFlow Gateway operando na porta 8080[39m
gateway-1 | [00:55:24] [32mINFO[39m (1): [36mincoming request[39m
gateway-1 | [35mreqId[39m: "req-1"
gateway-1 | [35mreq[39m: {
gateway-1 |   "method": "POST",
gateway-1 |   "url": "/api/streaming/play",
gateway-1 |   "host": "localhost:8080",
gateway-1 |   "remoteAddress": "172.18.0.1",
gateway-1 |   "remotePort": 52968
gateway-1 | }
gateway-1 | [00:55:24] [32mINFO[39m (1): [36mfetching from remote server[39m
gateway-1 | [35mreqId[39m: "req-1"
gateway-1 | [35msource[39m: "/streaming/play"
gateway-1 | [00:55:26] [32mINFO[39m (1): [36mresponse received[39m
gateway-1 | [35mreqId[39m: "req-1"
gateway-1 | [00:55:26] [32mINFO[39m (1): [36mrequest completed[39m
gateway-1 | [35mreqId[39m: "req-1"
gateway-1 | [35mres[39m: {
gateway-1 |   "statusCode": 201
gateway-1 | }
gateway-1 | [35mresponseTime[39m: 2649.878471999895
gateway-1 | [00:57:07] [32mINFO[39m (1): [36mincoming request[39m
gateway-1 | [35mreqId[39m: "req-2"
gateway-1 | [35mreq[39m: {
gateway-1 |   "method": "GET",
gateway-1 |   "url": "/api/analytics/report",
gateway-1 |   "host": "localhost:8080",
gateway-1 |   "remoteAddress": "172.18.0.1",
gateway-1 |   "remotePort": 49570
gateway-1 | }
gateway-1 | [00:57:07] [32mINFO[39m (1): [36mfetching from remote server[39m
gateway-1 | [35mreqId[39m: "req-2"
gateway-1 | [35msource[39m: "/analytics/report"
gateway-1 | [00:57:07] [32mINFO[39m (1): [36mresponse received[39m
gateway-1 | [35mreqId[39m: "req-2"
gateway-1 | [00:57:07] [32mINFO[39m (1): [36mrequest completed[39m
gateway-1 | [35mreqId[39m: "req-2"
gateway-1 | [35mres[39m: {
gateway-1 |   "statusCode": 200
gateway-1 | }
gateway-1 | [35mresponseTime[39m: 290.17822200013325
gateway-1 | [00:59:40] [32mINFO[39m (1): [36mincoming request[39m
gateway-1 | [35mreqId[39m: "req-3"
gateway-1 | [35mreq[39m: {
gateway-1 |   "method": "GET",
gateway-1 |   "url": "/health",
gateway-1 |   "host": "localhost:8080",
gateway-1 |   "remoteAddress": "172.18.0.1",
gateway-1 |   "remotePort": 47132
gateway-1 | }
gateway-1 | [00:59:40] [32mINFO[39m (1): [36mrequest completed[39m
gateway-1 | [35mreqId[39m: "req-3"
gateway-1 | [35mres[39m: {
gateway-1 |   "statusCode": 503
gateway-1 | }

```

Fonte: Do autor

Para compreender o comportamento da requisição, foi necessário analisar manualmente outros arquivos, como *logs_streaming_sem_notification.txt*, onde é possível identificar a falha ao tentar acessar o *notification-service* (*getaddrinfo ENOTFOUND*), além da conclusão da requisição com status 201 em aproximadamente 2627ms. Esse processo evidencia que o diagnóstico depende de correlação manual entre múltiplos arquivos de log, o que é inviável em cenários reais de incidente.

O *health check* também apresenta limitações. No arquivo *health_degraded.json*, o sistema indica estado "degraded", identificando o *recommendation-service* como indisponível. O log correspondente *logs_gateway_health_degraded.txt* (Figura 7) confirma a resposta com status 503. Entretanto, essas informações não permitem identificar quais requisições foram afetadas, quantos usuários sofreram impacto ou por quanto tempo o sistema permaneceu degradado.

Figura 7 – Log que Confirma o Estado Degradado no Gateway.

```

logs_gateway_health_degraded.txt - Notepad
File Edit Format View Help
gateway-1 | [00:55:11] [32mINFO[39m (1): [36mServer listening at http://172.0.0.1:8080[39m
gateway-1 | [00:55:11] [32mINFO[39m (1): [36mServer listening at http://172.18.0.9:8080[39m
gateway-1 | [00:55:11] [32mINFO[39m (1): [36mStreamFlow Gateway operando na porta 8080[39m
gateway-1 | [00:55:24] [32mINFO[39m (1): [36mincoming request[39m
gateway-1 | [35mreq[39m: {
gateway-1 |   "method": "POST",
gateway-1 |   "url": "/api/streaming/play",
gateway-1 |   "host": "localhost:8080",
gateway-1 |   "remoteAddress": "172.18.0.1",
gateway-1 |   "remotePort": 52968
gateway-1 | }
gateway-1 | [00:55:24] [32mINFO[39m (1): [36mfetching from remote server[39m
gateway-1 | [35mreq[39m: "req-1"
gateway-1 | [35msource[39m: "/streaming/play"
gateway-1 | [00:55:26] [32mINFO[39m (1): [36mresponse received[39m
gateway-1 | [35mreq[39m: "req-1"
gateway-1 | [00:55:26] [32mINFO[39m (1): [36mrequest completed[39m
gateway-1 | [35mres[39m: {
gateway-1 |   "statusCode": 201
gateway-1 | }
gateway-1 | [35mresponseTime[39m: 2649.870471999895
gateway-1 | [00:57:07] [32mINFO[39m (1): [36mincoming request[39m
gateway-1 | [35mreq[39m: "req-2"
gateway-1 | [35mreq[39m: {
gateway-1 |   "method": "GET",
gateway-1 |   "url": "/api/analytics/report",
gateway-1 |   "host": "localhost:8080",
gateway-1 |   "remoteAddress": "172.18.0.1",
gateway-1 |   "remotePort": 49570
gateway-1 | }
gateway-1 | [00:57:07] [32mINFO[39m (1): [36mfetching from remote server[39m
gateway-1 | [35mreq[39m: "req-2"
gateway-1 | [35msource[39m: "/analytics/report"
gateway-1 | [00:57:07] [32mINFO[39m (1): [36mresponse received[39m
gateway-1 | [35mreq[39m: "req-2"
gateway-1 | [00:57:07] [32mINFO[39m (1): [36mrequest completed[39m
gateway-1 | [35mres[39m: {
gateway-1 |   "statusCode": 200
gateway-1 | }
gateway-1 | [35mresponseTime[39m: 290.17822200013325
gateway-1 | [00:59:40] [32mINFO[39m (1): [36mincoming request[39m
gateway-1 | [35mreq[39m: "req-3"
gateway-1 | [35mreq[39m: {
gateway-1 |   "method": "GET",
gateway-1 |   "url": "/health",
gateway-1 |   "host": "localhost:8080",
gateway-1 |   "remoteAddress": "172.18.0.1",
gateway-1 |   "remotePort": 47132
gateway-1 | }
gateway-1 | [00:59:40] [32mINFO[39m (1): [36mrequest completed[39m
gateway-1 | [35mreq[39m: "req-3"
gateway-1 | [35mres[39m: {
gateway-1 |   "statusCode": 503
gateway-1 | }
gateway-1 | [35mresponseTime[39m: 67.6115489997901
gateway-1 | [01:00:00] [32mINFO[39m (1): [36mincoming request[39m

```

Fonte: Do autor

4.3 Causa Raiz e Lacunas de Observabilidade

O problema central é que o StreamFlow possui logging isolado por serviço, mas não possui observabilidade distribuída. Não existe um identificador global, como *correlation_id* ou *trace_id*, sendo propagado entre os serviços. Além disso, não há mecanismos de *tracing* distribuído nem coleta estruturada de métricas.

Como consequência, o sistema não consegue responder de forma objetiva perguntas fundamentais como: “onde ocorreu a falha”, “qual foi sua duração” e “qual foi o impacto no usuário”.

Para atingir esse nível de visibilidade, são necessárias cinco capacidades principais: correlação ponta a ponta entre requisições, rastreamento distribuído do fluxo, métricas técnicas (latência, erros, volume), métricas de negócio (usuários afetados, falhas de reprodução) e mecanismos de alerta e visualização.

4.4 Proposta de Melhoria

Recomenda-se a implementação de uma estratégia de observabilidade baseada em três pilares: logs centralizados, tracing distribuído e métricas agregadas.

No nível de logs, todos os serviços devem adotar um padrão comum de campos, incluindo *timestamp*, *service*, *level*, *correlation_id*, *method*, *url*, *status_code*, *response_time* e, quando possível, *user_id*. Esses logs devem ser enviados para uma plataforma centralizada, permitindo busca e correlação eficiente.

Para o rastreamento distribuído, recomenda-se a utilização de ferramentas como *OpenTelemetry*, permitindo que cada requisição iniciada no gateway receba um *trace_id* único, propagado automaticamente entre os serviços. Dessa forma, seria possível visualizar o fluxo completo de uma requisição, identificar gargalos e localizar falhas com precisão.

No que diz respeito às métricas, ferramentas como *Prometheus* e *Grafana* podem ser utilizadas para coletar e visualizar indicadores como latência por endpoint, taxa de erro, volume de requisições e disponibilidade dos serviços. Além disso, devem ser definidos dashboards e alertas para identificar rapidamente degradações no sistema.

Com a adoção dessas práticas, a equipe deixa de depender da análise manual de múltiplos arquivos de log e passa a contar com uma visão integrada do sistema. Isso permite identificar rapidamente a origem de falhas, medir sua duração e estimar o impacto no negócio, elevando significativamente a maturidade operacional da plataforma.

5. Consistência Eventual

“Recomendações mostrando conteúdo que já foi removido.”

A afirmação do gerente é correta. Esse problema pode ocorrer devido à forma como os dados são distribuídos entre serviços que representam contextos distintos, especialmente entre o *catalog-service* e o *recommendation-service*.

5.1 Análise do Estado Atual

O *catalog-service* é responsável pelo estado oficial dos conteúdos, armazenando informações como título, gênero e o campo *license_active*, que indica se o conteúdo está disponível para reprodução. Já o *recommendation-service* mantém dados próprios, como histórico de visualização e preferências dos usuários, armazenando principalmente identificadores de filmes (*movie_id*), sem possuir controle sobre o estado atual desses conteúdos.

Essa separação é esperada em uma arquitetura de microsserviços, pois cada serviço deve ser responsável por seu próprio domínio e base de dados. No entanto, ela introduz um risco de inconsistência entre os serviços.

5.2 Exemplo e Causa da Inconsistência

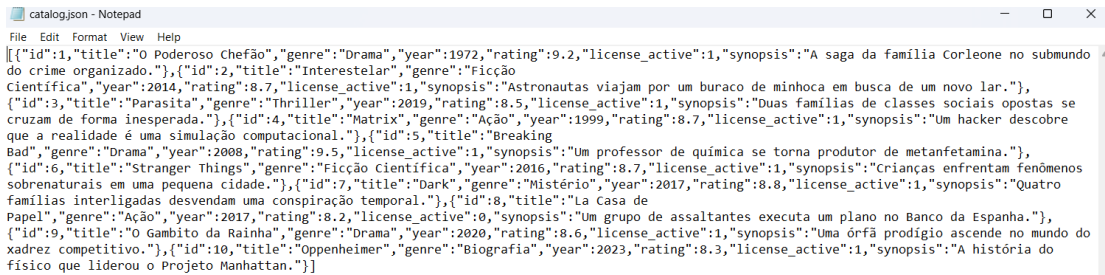
O ponto de inconsistência surge quando mudanças no catálogo não são refletidas no serviço de recomendação, devido à ausência de mecanismos de sincronização entre os serviços.

Exemplo:

1. O usuário assiste ao filme de id 8.
2. O *recommendation-service* registra esse id em "view_history".
3. Depois, o *catalog-service* remove esse filme ou marca sua licença como inativa.
4. O *recommendation-service* continua com o id antigo no histórico.
5. Uma recomendação pode continuar considerando esse conteúdo mesmo ele não estando mais disponível.

Esse cenário demonstra como dados podem ficar desatualizados entre serviços, resultando em recomendações inconsistentes para o usuário. A causa principal não está na separação dos bancos de dados, mas sim na ausência de propagação de mudanças do catálogo para o serviço de recomendação. As Figuras 8, 9 e 10 ilustram as arquiteturas citadas.

Figura 8 – Log dos conteúdos do Catálogo e o Campo *license_active*.



```

[{"id":1,"title":"O Poderoso Chefão","genre":"Drama","year":1972,"rating":9.2,"license_active":1,"synopsis":"A saga da família Corleone no submundo do crime organizado."}, {"id":2,"title":"Interestelar","genre":"Ficção Científica","year":2014,"rating":8.7,"license_active":1,"synopsis":"Astronautas viajam por um buraco de minhoca em busca de um novo lar."}, {"id":3,"title":"Parasita","genre":"Thriller","year":2019,"rating":8.5,"license_active":1,"synopsis":"Duas famílias de classes sociais opostas se cruzam de forma inesperada."}, {"id":4,"title":"Matrix","genre":"Ação","year":1999,"rating":8.7,"license_active":1,"synopsis":"Um hacker descobre que a realidade é uma simulação computacional."}, {"id":5,"title":"Breaking Bad","genre":"Drama","year":2008,"rating":9.5,"license_active":1,"synopsis":"Um professor de química se torna produtor de metanfetamina."}, {"id":6,"title":"Stranger Things","genre":"Ficção Científica","year":2016,"rating":8.7,"license_active":1,"synopsis":"Crianças enfrentam fenômenos sobrenaturais em uma pequena cidade."}, {"id":7,"title":"Dark","genre":"Mistério","year":2017,"rating":8.8,"license_active":1,"synopsis":"Quatro famílias interligadas desvendam uma conspiração temporal."}, {"id":8,"title":"La Casa de Papel","genre":"Ação","year":2017,"rating":8.2,"license_active":0,"synopsis":"Um grupo de assaltantes executa um plano no Banco da Espanha."}, {"id":9,"title":"O Gambito da Rainha","genre":"Drama","year":2020,"rating":8.6,"license_active":1,"synopsis":"Uma órfã prodígio ascende no mundo do xadrez competitivo."}, {"id":10,"title":"Oppenheimer","genre":"Biografia","year":2023,"rating":8.3,"license_active":1,"synopsis":"A história do físico que liderou o Projeto Manhattan."}]

```

Fonte: Do autor

Figura 9 – Método GET para Verificar Licença no Catálogo.

```

// GET /catalog/:id/license – verificar licença (usado pelo streaming-service)
app.get('/catalog/:id/license', async (request, reply) => {
  const movie = db.prepare('SELECT id, title, license_active FROM movies WHERE id = ?').get(request.params.id);
  if (!movie) {
    return reply.code(404).send({ error: 'Título não encontrado.' });
  }
  return { id: movie.id, title: movie.title, licensed: movie.license_active === 1 };
});

```

Fonte: Do autor

Figura 10 – Método GET de Histórico de Recomendações.

```

// GET /recommendations/:userId – recomendações para o usuário
app.get('/recommendations/:userId', async (request, reply) => {
  const { userId } = request.params;

  const prefs = db.prepare('SELECT * FROM user_preferences WHERE user_id = ?').get(userId);
  const history = db.prepare(
    'SELECT movie_id FROM view_history WHERE user_id = ? ORDER BY viewed_at DESC LIMIT 20'
  ).all(userId);

  return {
    userId,
    preferences: prefs ? JSON.parse(prefs.preferred_genres) : [],
    recentlyViewed: history.map(h => h.movie_id),
    // Recomendação simplificada: retorna gêneros preferidos
    suggestedGenres: prefs ? JSON.parse(prefs.preferred_genres) : ['Drama', 'Ação'],
  };
});

```

Fonte: Do autor

Dessa forma, o *recommendation-service* passa a operar com dados potencialmente obsoletos por tempo indefinido, caracterizando um caso de consistência eventual não controlada, onde não há garantia de quando (ou se) a informação será sincronizada.

5.3 Proposta de Melhoria e Níveis de Consistência

Para mitigar os problemas identificados, recomenda-se a adoção de uma estratégia baseada em eventos para sincronização entre serviços. É recomendado que o *catalog-service* publique eventos sempre que houver mudanças importantes no catálogo. Exemplos de eventos:

- "MovieRemoved": quando um conteúdo for removido;
- "LicenseExpired": quando a licença de um conteúdo expirar;
- "CatalogUpdated": quando dados importantes do catálogo forem alterados.

O *recommendation-service* deve consumir esses eventos e atualizar sua base interna, removendo ou invalidando conteúdos indisponíveis. Além disso, pode ser aplicada uma validação no momento de exibir recomendações, garantindo que o conteúdo ainda exista e esteja licenciado.

Entretanto, nem todos os domínios do sistema exigem o mesmo nível de consistência. É necessário definir claramente onde a consistência deve ser forte e onde pode ser eventual, conforme apresentado na Tabela 2.

Tabela 2 – Níveis de Consistência por Domínio no StreamFlow

Domínio	Tipo de Consistência	Justificativa
Play / Licença	Forte	Necessário validar disponibilidade antes da reprodução
Catálogo exibido	Forte	Usuário não deve visualizar conteúdo indisponível
Billing / Faturamento	Forte	Envolve dados financeiros e cobrança
Histórico de visualização	Eventual	Pode ser atualizado com pequeno atraso
Recomendações	Eventual	Pode tolerar atraso, mas deve ser sincronizado com eventos do catálogo
Notificações	Eventual	Não impacta diretamente a experiência principal
Analytics / Relatórios	Eventual	Pode trabalhar com dados defasados, desde que identificado

Fonte: Do autor

Dessa forma, a solução não está em centralizar dados ou exigir atualização imediata entre todos os serviços, mas sim em combinar sincronização orientada a eventos com a definição adequada de níveis de consistência, equilibrando desempenho e confiabilidade.

6. Lei de Conway

“Mesma equipe de 15 pessoas operando 7 serviços.”

A afirmação do gerente é correta. A proporção entre o tamanho da equipe e a quantidade de microsserviços evidencia um desalinhamento entre a arquitetura do sistema e a estrutura organizacional responsável por mantê-lo.

6.1 Análise do Cenário Atual

A StreamFlow possui 15 pessoas responsáveis por 7 microsserviços, além do API Gateway e toda a infraestrutura associada, incluindo containers, bancos de dados, pipelines de CI/CD, monitoramento, logs e tratamento de incidentes. Esse cenário exige não apenas desenvolvimento, mas também forte capacidade operacional.

De acordo com o princípio conhecido como Lei de Conway, a arquitetura de um sistema tende a refletir a estrutura da organização que o desenvolve. Em contextos onde há múltiplos serviços independentes, espera-se que existam equipes ou grupos claramente responsáveis por cada domínio. No entanto, quando há muitos serviços e uma equipe relativamente pequena, a complexidade arquitetural supera a capacidade organizacional de gerenciá-la.

Além disso, cada serviço exige conhecimento específico. Por exemplo, compreender o fluxo de reprodução (play) envolve múltiplos serviços, enquanto a análise de relatórios exige entendimento do domínio financeiro e do acoplamento entre serviços de dados. Isso aumenta a dependência de conhecimento especializado.

6.2 Impacto da Distribuição de Conhecimento

A saída de dois tech leads agravou significativamente o cenário. O conhecimento técnico e operacional estava concentrado nessas pessoas, o que indica baixa disseminação interna. Como resultado, a equipe passou a ter dificuldade para diagnosticar problemas e compreender o funcionamento completo do sistema.

Esse cenário evidencia três problemas principais:

- excesso de serviços para o tamanho da equipe;
- concentração de conhecimento crítico em poucos membros;
- ausência de mecanismos (como observabilidade madura) para compensar essa complexidade.

A combinação desses fatores torna a operação do sistema mais frágil e dependente de indivíduos específicos.

6.3 Proposta de Reorganização e Ações Recomendadas

Para alinhar a arquitetura à capacidade da equipe, recomenda-se uma reorganização baseada em áreas de responsabilidade, em vez de manter uma divisão rígida por microserviço. Uma estrutura mais adequada para o contexto da StreamFlow seria organizar a equipe em três frentes principais:

1. **Plataforma e Observabilidade:** Responsável pelo API Gateway, infraestrutura, logs, métricas, tracing, CI/CD e padronização operacional.
2. **Produto de Streaming:** Focada na experiência do usuário, incluindo catálogo, reprodução, recomendações e notificações.
3. **Financeiro e Dados:** Responsável por faturamento, assinaturas, relatórios e analytics.

Essa divisão permite organizar o trabalho por domínio de negócio e operação, reduzindo a dependência de conhecimento isolado e facilitando a colaboração entre os membros da equipe. Para uma equipe de 15 pessoas, essa abordagem é mais realista do que manter múltiplos serviços como se cada um tivesse uma equipe dedicada.

Além da reorganização estrutural, é necessário simplificar a própria arquitetura. Nesse sentido, recomenda-se:

- Manter serviços com domínio bem definido, como autenticação, catálogo, reprodução e faturamento.
- Consolidar o *analytics-service* ao domínio financeiro, reduzindo acoplamentos desnecessários.
- Remover o *notification-service* do fluxo síncrono, tratando-o como processo assíncrono.
- Fortalecer a camada de plataforma, garantindo observabilidade, padronização e documentação adequadas.

Com essas mudanças, a arquitetura passa a refletir melhor a estrutura da equipe, reduzindo a complexidade operacional e tornando o sistema mais sustentável. Dessa forma, o StreamFlow se aproxima de um modelo mais equilibrado, no qual a organização consegue

efetivamente operar a arquitetura que construiu, evitando dependência excessiva de conhecimento especializado e melhorando sua capacidade de resposta a incidentes.

7. O Caso Prime Video

“O concorrente fez melhor com menos.”

A afirmação do gerente é parcialmente correta. O caso do Prime Video não implica que a arquitetura de microsserviços deva ser abandonada, mas evidencia que pipelines excessivamente fragmentados e com alto custo operacional podem se tornar mais eficientes quando consolidados de forma estratégica.

7.1 Análise do Cenário no StreamFlow

O serviço que mais se aproxima desse cenário é o *analytics-service*. Ele opera como um pipeline sequencial interno, composto por etapas de coleta, agregação e formatação de dados. Esse comportamento é evidenciado nos logs, onde o fluxo é executado de forma linear e a requisição é concluída com sucesso em aproximadamente 278ms.

Além disso, os dados mostram que, mesmo com o *billing-service* indisponível, o *analytics-service* continua gerando relatórios contendo informações como número de assinantes, receita recorrente, faturas e distribuição de planos. Isso demonstra que o serviço não depende da API do faturamento, mas acessa diretamente o mesmo banco de dados.

7.2 Causa Raiz

O problema central é que o *analytics-service* não possui autonomia real como microsserviço. Apesar de estar separado, ele depende diretamente das estruturas internas do domínio financeiro, acessando tabelas como *subscriptions* e *invoices*. Isso cria um acoplamento forte, onde mudanças no banco de dados podem quebrar o serviço sem qualquer alteração em interfaces públicas.

Nesse cenário, existe o custo de manter um microsserviço independente — como deploy, monitoramento e infraestrutura — sem o principal benefício esperado, que é a independência entre domínios. Além disso, sua natureza de pipeline sequencial reforça que ele não se beneficia de distribuição como serviço isolado.

7.3 Proposta de Melhoria

Diante disso, a principal ação recomendada é a consolidação do *analytics-service* ao domínio de faturamento. Isso não significa eliminar a funcionalidade de analytics, mas integrá-la ao *billing-service*, que já possui os dados necessários. A geração de relatórios pode ser implementada como rotinas internas, endpoints específicos ou processos assíncronos, sem a necessidade de manter um serviço separado artificialmente.

Outros serviços também apresentam oportunidades de melhoria, mas não necessariamente de consolidação. O *notification-service*, por exemplo, demonstrou não ser essencial no fluxo de reprodução. Testes indicam que, mesmo com falha nesse serviço, a reprodução ocorre normalmente, evidenciando que notificações são efeitos colaterais e não pré-condições.

Nesse caso, a solução adequada é torná-lo assíncrono, permitindo que o *streaming-service* publique eventos como *PlaybackStarted*, que seriam processados em segundo plano.

De forma semelhante, o *recommendation-service* possui um domínio válido e independente, relacionado a preferências e histórico de usuários. O problema não está na sua existência como microsserviço, mas no fato de estar inserido no caminho síncrono do *play*. A melhoria recomendada é tornar o registro de visualizações assíncrono, sem necessidade de consolidar o serviço.

Portanto, o caso do Prime Video não sugere a eliminação dos microsserviços, mas sim a necessidade de avaliar criticamente onde a separação realmente agrega valor. No contexto do StreamFlow, o principal candidato à consolidação é o *analytics-service*, por apresentar alto acoplamento e baixa autonomia. Já os demais serviços devem ser ajustados para reduzir dependências síncronas desnecessárias, mantendo a separação apenas onde ela contribui efetivamente para a arquitetura.

8. Refatoração do Fluxo de Play com Molecular

8.1 Introdução ao Molecular e Motivação

O Molecular é um framework para Node.js voltado ao desenvolvimento de arquiteturas baseadas em microsserviços. Seu objetivo é simplificar a comunicação entre serviços distribuídos, abstraindo parte da complexidade relacionada à rede e à coordenação entre componentes da aplicação. Para isso, o framework fornece mecanismos como service discovery, circuit breaker, retry automático, timeout, load balancing, métricas e comunicação orientada a eventos através de um broker distribuído.

Conforme discutido na Seção 3, o principal problema identificado no fluxo de reprodução estava na execução síncrona de operações que não eram essenciais para iniciar o Play. Embora a validação de licença realizada pelo *catalog-service* seja obrigatória, operações como o registro de visualização no *recommendation-service* e o envio de notificações pelo *notification-service* representam apenas efeitos colaterais do processo, não devendo bloquear a resposta ao usuário.

Com base nesse diagnóstico, a atividade bônus foi desenvolvida utilizando o Molecular com o objetivo de reduzir o acoplamento causado pelas chamadas HTTP síncronas entre serviços.

8.2 Implementação dos Eventos

A refatoração concentrou-se principalmente no *streaming-service*, responsável pela orquestração do fluxo de reprodução. Inicialmente, esse serviço realizava chamadas síncronas utilizando *await fetch* para o *recommendation-service* e para o *notification-service*, aguardando a conclusão dessas operações antes de responder ao cliente.

Após a adoção do Molecular, essas chamadas foram substituídas pela emissão do evento *playback.started* através do broker distribuído. A comunicação entre os serviços foi realizada utilizando o transporter TCP do Molecular, permitindo a troca de eventos entre os containers Docker da aplicação sem chamadas HTTP explícitas entre os microsserviços. A figura 11 apresenta a emissão do evento pelo *streaming-service*.

Figura 11 – Emissão do evento playback.started no streaming-service.

```
// — PASSO 2 & 3: Gera evento para registrar recomendação e notificar início da sessão
const broker = new ServiceBroker({
  logger: console,
  transporter: "TCP",
});

broker.emit("playback.started", {
  userId,
  type: "session_started",
  message: `Reprodução iniciada: ${licenseData.title}`,
  movieId,
});

app.log.info("Evento playback.started emitido");
```

Fonte: Do autor

Nos serviços consumidores, os eventos passaram a ser processados através da propriedade events do Moleculer.

Figura 12 – Processamento do evento playback.started no recommendation-service.

```
broker.createService({
  name: "viewed",
  events: {
    "playback.started": async (ctx) => {
      const { userId, movieId } = ctx.params || {};

      if (!userId || !movieId) {
        app.log.warn("userId e message são obrigatórios.");
        return;
      }

      db.prepare(
        "INSERT INTO view_history (user_id, movie_id) VALUES (?, ?)",
      ).run(userId, movieId);

      app.log.info("Recomendação enviada");

      return { registered: true, userId, movieId };
    },
  },
});
```

Fonte: Do autor

Com essa abordagem, o *recommendation-service* e o *notification-service* deixam de bloquear o fluxo principal do Play, permitindo que o *streaming-service* responda ao usuário imediatamente após a validação da licença e criação da sessão local.

8.3 Avaliação do Desempenho

Os resultados obtidos na Seção 3 foram utilizados como base de comparação com a nova

implementação. Na versão original do sistema, o endpoint POST /streaming/play apresentou tempo médio de aproximadamente 259 ms devido à cadeia de chamadas síncronas.

Após a substituição das chamadas síncronas não essenciais pela emissão assíncrona de eventos utilizando Moleculer, a latência observada caiu para aproximadamente 22 ms nos testes realizados em ambiente Docker, representando uma redução próxima de 92% no tempo de resposta do fluxo de Play.

Para validar essa redução, foram realizadas 10 execuções do endpoint POST /streaming/play utilizando *curl* com registro automático do tempo total da requisição através da opção *%{time_total}*. Os resultados foram registrados no arquivo *tempos_moleculer.txt*, apresentando média de aproximadamente 0.0219168 s, equivalente a 21,91 ms.

8.4 Limitações da Abordagem

Apesar dos ganhos de desempenho obtidos com a comunicação orientada a eventos, observou-se que a complexidade da comunicação distribuída não desaparece, passando apenas a ser abstraída pelo framework. Durante a implementação, verificou-se maior dificuldade para rastrear falhas e acompanhar o fluxo de execução, já que erros em eventos dependem fortemente de mecanismos adequados de logging e observabilidade.

Diante disso, optou-se por uma adoção parcial do Moleculer, utilizando-o principalmente para processamento assíncrono de eventos fora do fluxo crítico. Uma migração completa introduziria complexidade adicional desnecessária e dificultaria atividades de debug e manutenção do sistema.

REFERÊNCIAS

RED HAT. O que são microsserviços? Disponível em: <https://www.redhat.com/pt-br/topics/microservices/what-are-microservices>. Acesso em: 1 maio 2026. Citado na página 6.

HOSTINGER. O que é Docker Compose? Disponível em: <https://www.hostinger.com/br/tutoriais/o-que-e-docker-compose>. Acesso em: 1 maio 2026. Citado na página 6.